POINT-TO-POINT COMMUNICATION

Primitives	Semantics
MPI_Send	Blocking (might block or buffer), asynchronous
MPI_Isend	Nonblocking, asynchronous
MPI_Ssend	Blocking, synchronous
MPI_Recv	Blocking
MPI_Irecv	Non-blocking
MPI_Sendrecv	Send a message and post a (blocking) receive before blocking
MPI_Wait	Wait until an asynchronous call is completed

- MPI_Sendrecv is very useful, take it into consideration when you have to send and receive something at the same time.

COLLECTIVE COMMUNICATION

- Barrier -> MPI_Barrier(MPI_COMM_WORLD), Creates a barrier for all processes, no process is allowed to continue before all processes have reached the barrier
- Broadcast -> MPI_Bcast(void * data, int count, MPI_Datatype type, int root, MPI_Comm communicator), invoked both by a root process and a receiver process
- Reduce -> MPI_Reduce(void* send_data, void* recv_data, int count, MPI_Datatype type, MPI_Op op, int root, MPI_Comm communicatot)
    - send_data variable is an array of elements that each process wants to reduce
    - recv_data variable in the root node contains the reduced data
- AllReduce -> it has the same signature as MPI_Reduce but without the root node, it stores the reduced data at every node.

Reduce operators

MPI_MAX	Returns the maximum element
MPI_MIN	Returns the minimum element
MPI_SUM	Sums the elements
MPI_PROD	Multiplies the elements
MPI_LAND	Performs the logical *and* across the elements
MPI_LOR	Performs the logical *or* across the elements
MPI_BAND	Performs a bitwise *and* across the bits of the elements
MPI_BOR	Performs a bitwise *or* across the bits of the elements
MPI_MAXLOC	Returns the maximum value and the rank of the process that owns it
MPI_MINLOC	Returns the minimum value and the rank of the process that owns it

**It's also possible to create custom operators through the procedure MPI_Op_create

- MPI_Scatter(void* send_data, int send_count, MPI_Datatype send_type, void* recv_data, int recv_count, MPI_Datatype recv_datatype, int root, MPI_Comm communicator)

- MPI_Gather(void* send_data, int send_count, MPI_Datatype send_datatype, void* recv_data,

int recv_count, MPI_Datatype recv_datatype, int root, MPI_Comm communicator)